

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

Ansible Workshop Protocol

Ausgeführt von: David Veigel

Kevin Forter

Personenkennzeichen: 2310257030,
2500257003

BegutachterIn:

Wien, den 18. Dezember 2025

Inhaltsverzeichnis

1	Ansible Workshop	1
1.1	Infrastructure Setup (AWS)	1
1.1.1	Terraform Configuration	1
1.1.2	Execution	2
1.2	Enable SSH Trust (Master to Nodes)	3
1.2.1	Generate Keys on Master	3
1.2.2	Distribute Keys to Workers	3
1.2.3	Test Connection	4
1.3	Install and Configure Ansible	4
1.3.1	Install Ansible	5
1.3.2	Create Workspace and Configuration	5
1.3.3	Create Inventory File	6
1.3.4	Verify Setup	6
1.4	Groups and Ad-Hoc Commands	7
1.4.1	Modify Inventory for Groups	7
1.4.2	Get MAC Addresses	7
1.4.3	Create and Copy File	7
1.5	Playbooks (The Webserver)	9
1.5.1	Create Index File	9
1.5.2	Create Playbook	10
1.5.3	Run Playbook	11
1.5.4	Verify Deployment	12
2	Ansible Workshop 2	12
2.1	Prerequisites Check	13
2.2	Refactor Webserver to a Role	13
2.3	Create the MySQL Role	14
2.4	Create the Main Playbook	15
2.5	Create Tests (TDD)	16
2.6	Execution & Verification	17

1 Ansible Workshop

This tutorial guides you step-by-step through setting up an Ansible environment on AWS EC2, configuring SSH trust, running ad-hoc commands, and writing a playbook to deploy a web server.

1.1 Infrastructure Setup (AWS)

Instead of manually creating instances via the AWS Console, we use Terraform to automate the provisioning of our infrastructure. This ensures reproducibility and consistency.

1.1.1 Terraform Configuration

We define our infrastructure in a `main.tf` file.

First, we configure the AWS provider and dynamically fetch the latest Ubuntu 22.04 LTS AMI.

```
provider "aws" {
  region = var.aws_region
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name = "name"
    values = [
      "ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*",
      "ubuntu/images/hvm-ssd/ubuntu-*22.04-amd64-server-*"
    ]
  }
  filter {
    name     = "virtualization-type"
    values   = ["hvm"]
  }
}
```

Next, we define a security group that allows SSH (22), HTTP (80), and HTTPS (443) traffic from anywhere.

```
resource "aws_security_group" "web_sg" {
  name = "lab-web-sg"
```

```

description = "Allow_SSH,_HTTP,_HTTPS"
vpc_id      = data.aws_vpc.default.id

ingress {
  description = "SSH"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}
# ... (HTTP and HTTPS rules similar to SSH)
}

```

We then provision 3 EC2 instances using the `count` parameter. We use the `t3.micro` instance type and the `vockey` key pair.

```

resource "aws_instance" "ubuntu_lab" {
  count          = 3
  ami            = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  key_name       = "vockey"

  vpc_security_group_ids = [aws_security_group.web_sg.id]

  tags = {
    Name = "ubuntu-lab-${count.index+_1}"
  }
}

```

Finally, we output the public IPs of the created instances to easily connect to them.

```

output "instance_public_ips" {
  description = "Public_IPs_of_the_3_Ubuntu_instances"
  value       = [for i in aws_instance.ubuntu_lab : i.public_ip]
}

```

1.1.2 Execution

To apply this configuration, we initialize Terraform and apply the plan.

```

terraform init
terraform apply

```

After the apply is complete, we receive the public IPs of our 3 instances (Ansible-Master, Worker-1, Worker-2) and can proceed to connect to them.

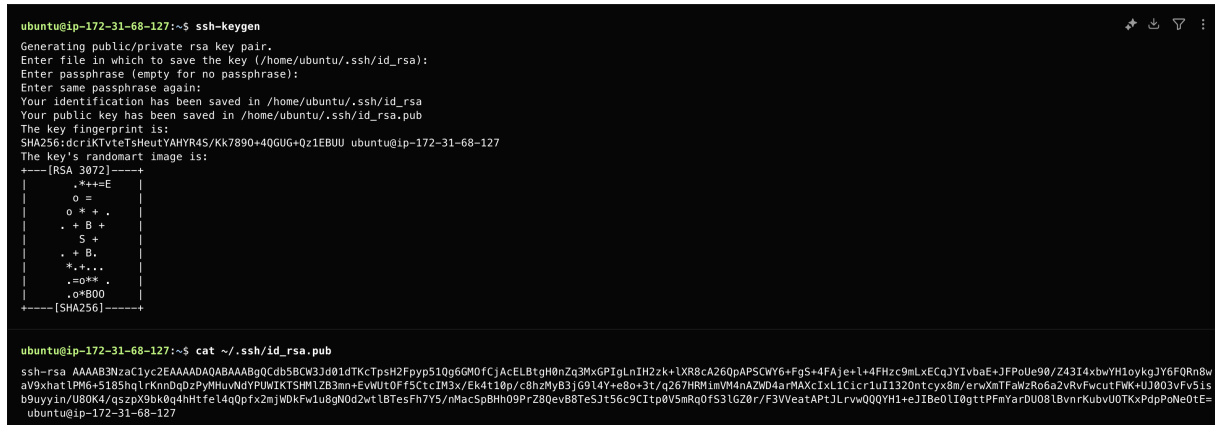
1.2 Enable SSH Trust (Master to Nodes)

To allow the Master to SSH into the Workers without a password, we configure SSH trust.

1.2.1 Generate Keys on Master

On the Master Instance, we generate a new SSH key pair.

```
ssh-keygen
cat ~/.ssh/id_rsa.pub
```



```
ubuntu@ip-172-31-68-127:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_rsa
Your public key has been saved in /home/ubuntu/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:dcrlKtvtTShuTtYAHYR4S/Kk7890+4QGUG+Qz1EBUU ubuntu@ip-172-31-68-127
The key's randomart image is:
+----[RSA 3072]-----+
|
|..*+..E
|o+..
|..+B+
|S+
|..+B.
|*+...
|..o+..
|..o+B00
|..
+----[SHA256]-----+

ubuntu@ip-172-31-68-127:~$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGBgcqdb5BCW3Jd01dTKcTpsH2Fpyp510q6GM0fCjAcELBtgh0nZq3MxGPiGlnIH2zk+LXR8cA260pAPSQWY6+FGS+4FAje+l+4FHzc9mLxEcQJYIvbaE+JFP0Ue90/Z4314xbwYH1oykgJY6FQR8w
aV9xhatlPM6+5185hqlrKnnDqDzPyMHuVndYPUWIKTSHMLZB3mn+EvWU0f5CtcIM3x/Ek4t10p/c8hzMyB3jG914Y+e8o+3t/q267HRMimVM4nAZnD4arMAXcIxL1Cicr1uI1320ntcyx8m/erwXnTFaWzRo6a2vRvFwcutFWK+UJ003vFv5is
b9uyyin/U80K4/qszpX9bk8q4hHtfe14q0pfXzmjWdKfWu8gN0d2wtlBTesFh7Y5/nMacSpBh09PrZ80evB8Te5t56c9CIt0V5mRq0f531GZ0r/F3VVeatAPTJLrvwQQYH1+eJIBe0LI0gttPFmYarDU08lBvnrKubvU0TKxPdpPoNe0tE=
ubuntu@ip-172-31-68-127
```

Abbildung 1: Generating SSH Keys on Master

1.2.2 Distribute Keys to Workers

On both Worker-1 and Worker-2, we append the public key from the Master to the `authorized_keys` file.

```
cat >> ~/.ssh/authorized_keys
# Paste key here, Enter, Ctrl+D
```



```
ubuntu@ip-172-31-69-83:~$ cat >> ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGBgcqdb5BCW3Jd01dTKcTpsH2Fpyp510q6GM0fCjAcELBtgh0nZq3MxGPiGlnIH2zk+LXR8cA260pAPSQWY6+FGS+4FAje+l+4FHzc9mLxEcQJYIvbaE+JFP0Ue90/Z4314xbwYH1oykgJY6FQR8w
aV9xhatlPM6+5185hqlrKnnDqDzPyMHuVndYPUWIKTSHMLZB3mn+EvWU0f5CtcIM3x/Ek4t10p/c8hzMyB3jG914Y+e8o+3t/q267HRMimVM4nAZnD4arMAXcIxL1Cicr1uI1320ntcyx8m/erwXnTFaWzRo6a2vRvFwcutFWK+UJ003vFv5is
b9uyyin/U80K4/qszpX9bk8q4hHtfe14q0pfXzmjWdKfWu8gN0d2wtlBTesFh7Y5/nMacSpBh09PrZ80evB8Te5t56c9CIt0V5mRq0f531GZ0r/F3VVeatAPTJLrvwQQYH1+eJIBe0LI0gttPFmYarDU08lBvnrKubvU0TKxPdpPoNe0tE=
ubuntu@ip-172-31-68-127

ubuntu@ip-172-31-69-83:~$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGBgcqdb5BCW3Jd01dTKcTpsH2Fpyp510q6GM0fCjAcELBtgh0nZq3MxGPiGlnIH2zk+LXR8cA260pAPSQWY6+FGS+4FAje+l+4FHzc9mLxEcQJYIvbaE+JFP0Ue90/Z4314xbwYH1oykgJY6FQR8w
aV9xhatlPM6+5185hqlrKnnDqDzPyMHuVndYPUWIKTSHMLZB3mn+EvWU0f5CtcIM3x/Ek4t10p/c8hzMyB3jG914Y+e8o+3t/q267HRMimVM4nAZnD4arMAXcIxL1Cicr1uI1320ntcyx8m/erwXnTFaWzRo6a2vRvFwcutFWK+UJ003vFv5is
b9uyyin/U80K4/qszpX9bk8q4hHtfe14q0pfXzmjWdKfWu8gN0d2wtlBTesFh7Y5/nMacSpBh09PrZ80evB8Te5t56c9CIt0V5mRq0f531GZ0r/F3VVeatAPTJLrvwQQYH1+eJIBe0LI0gttPFmYarDU08lBvnrKubvU0TKxPdpPoNe0tE=
ubuntu@ip-172-31-68-127
```

Abbildung 2: Adding Master Key to Worker

1.2.3 Test Connection

From the Master, we verify the connection to the workers using their Private IPs.

```
ssh ubuntu@<PRIVATE_IP_OF_WORKER_1>
```

```
ubuntu@ip-172-31-68-127:~$ ssh ubuntu@172.31.73.159
The authenticity of host '172.31.73.159 (172.31.73.159)' can't be established.
ED25519 key fingerprint is SHA256:J3uBYWwV081yFoJNWUoi4Z8AocsIgWE140Xif7vmHg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.31.73.159' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1043-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Dec  5 13:55:54 UTC 2025

System load:  0.08      Processes:      104
Usage of /:   23.5% of 7.57GB   Users logged in:  1
Memory usage: 23%      IP4 address for ens5: 172.31.73.159
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Dec  5 13:43:54 2025 from 195.245.225.253
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-73-159:~$ exit
logout
Connection to 172.31.73.159 closed.
```

Abbildung 3: Successful Connection to Worker 1

```
ubuntu@ip-172-31-68-127:~$ ssh ubuntu@172.31.69.83
The authenticity of host '172.31.69.83 (172.31.69.83)' can't be established.
ED25519 key fingerprint is SHA256:UseIpZu9oP980pCvEG/vI+yz8AKKc4Lb0tkJvV2ysE.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.31.69.83' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1043-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Dec  5 13:56:33 UTC 2025

System load:  0.0      Processes:      104
Usage of /:   23.5% of 7.57GB   Users logged in:  1
Memory usage: 26%      IP4 address for ens5: 172.31.69.83
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Dec  5 13:45:21 2025 from 195.245.225.253
ubuntu@ip-172-31-69-83:~$ exit
logout
Connection to 172.31.69.83 closed.
```

Abbildung 4: Successful Connection to Worker 2

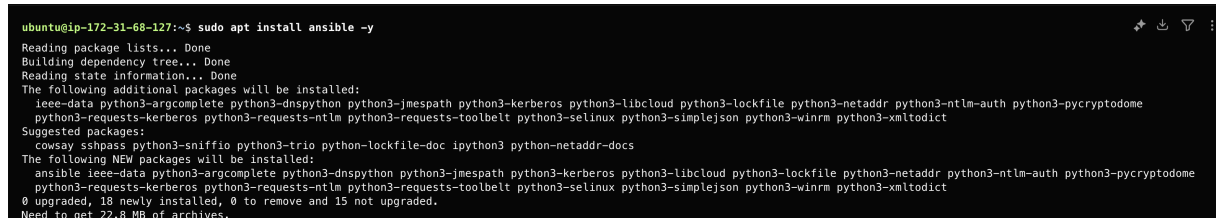
1.3 Install and Configure Ansible

These steps are performed on the Master instance only.

1.3.1 Install Ansible

We update the package list and install Ansible.

```
sudo apt update
sudo apt install ansible -y
```



```
ubuntu@ip-172-31-68-127:~$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ieee-data python3-argcomplete python3-dnspython python3-jmespath python3-kerberos python3-libcloud python3-lockfile python3-netaddr python3-ntlm-auth python3-pycryptodome
  python3-requests-kerberos python3-requests-ntlm python3-requests-toolbelt python3-selinux python3-simplejson python3-winrm python3-xlrd python3-xmldict
Suggested packages:
  cowsay sshpass python3-sniffio python3-trio python-lockfile-doc ipython3 python-netaddr-docs
The following NEW packages will be installed:
  ansible ieee-data python3-argcomplete python3-dnspython python3-jmespath python3-kerberos python3-libcloud python3-lockfile python3-netaddr python3-ntlm-auth python3-pycryptodome
  python3-requests-kerberos python3-requests-ntlm python3-requests-toolbelt python3-selinux python3-simplejson python3-winrm python3-xlrd python3-xmldict
0 upgraded, 18 newly installed, 0 to remove and 15 not upgraded.
Need to get 22.8 MB of archives.
```

Abbildung 5: Installing Ansible

1.3.2 Create Workspace and Configuration

We create a workspace directory and the `ansible.cfg` file.

```
mkdir ansible-lab
cd ansible-lab
nano ansible.cfg
```

The content of `ansible.cfg`:

```
[defaults]
inventory = hosts
host_key_checking = False
```



```
ubuntu@ip-172-31-68-127:~$ mkdir ansible-lab
ubuntu@ip-172-31-68-127:~$ cd ansible-lab
ubuntu@ip-172-31-68-127:~/ansible-lab$ nano ansible.cfg
ubuntu@ip-172-31-68-127:~/ansible-lab$
```

Abbildung 6: Creating Ansible Config

```
Master
GNU nano 6.2 ansible.cfg *
[defaults]
inventory = hosts
host_key_checking = False
```

Abbildung 7: Ansible Configuration Content

1.3.3 Create Inventory File

We create the `hosts` file to categorize our servers.

```
nano hosts
```

Initial content of `hosts`:

```
[servers]
server1 ansible_host=172.31.xx.xx
server2 ansible_host=172.31.xx.xx
```

```
Master
GNU nano 6.2 hosts *
[servers1]
server1 ansible_host=172.31.73.159

[servers2]
server2 ansible_host=172.31.69.83

[servers:children]
servers1
servers2
```

Abbildung 8: Hosts Inventory File

1.3.4 Verify Setup

We check the inventory list and test connectivity using the ping module.

```
ansible all --list-hosts
ansible all -m ping -o
```

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible all -m ping -o
server2 | SUCCESS => {"ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},"changed": false,"ping": "pong"}
server1 | SUCCESS => {"ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},"changed": false,"ping": "pong"}
```

Abbildung 9: Ping Pong Success

1.4 Groups and Ad-Hoc Commands

1.4.1 Modify Inventory for Groups

We update the `hosts` file to split the servers into groups.

```
[servers1]
server1 ansible_host=<WORKER_1_PRIVATE_IP>

[servers2]
server2 ansible_host=<WORKER_2_PRIVATE_IP>

[servers:children]
servers1
servers2
```

1.4.2 Get MAC Addresses

We use the `setup` module to gather facts, including MAC addresses.

```
ansible all -m setup
```

1.4.3 Create and Copy File

We create a dummy file on the Master and copy it to the `servers1` group.

```
echo "This_is_a_test_file" > testfile.txt
ansible servers1 -m copy -a "src=testfile.txt_dest=/home/ubuntu/testfile.txt
_mode=600"
```

```

ubuntu@ip-172-31-68-127:~/ansible-lab$ echo "This is a test file" > testfile.txt

ubuntu@ip-172-31-68-127:~/ansible-lab$ ls -la
total 20
drwxrwxr-x 2 ubuntu ubuntu 4096 Dec  5 14:15 .
drwxr-x--- 7 ubuntu ubuntu 4096 Dec  5 14:10 ..
-rw-rw-r-- 1 ubuntu ubuntu  55 Dec  5 14:07 ansible.cfg
-rw-rw-r-- 1 ubuntu ubuntu 130 Dec  5 14:13 hosts
-rw-rw-r-- 1 ubuntu ubuntu  20 Dec  5 14:15 testfile.txt

ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible servers1 -m copy -a "src=testfile.txt dest=/home/ubuntu/testfile.txt mode=600"
server1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": true,
  "checksum": "b56df8ed5365fca1419818aa384ba3b5e7756047",
  "dest": "/home/ubuntu/testfile.txt",
  "gid": 1000,
  "group": "ubuntu",
  "md5sum": "5dd39cab1c53c2c77cd352983f9641e1",
  "mode": "0600",
  "owner": "ubuntu",
  "size": 20,
  "src": "/home/ubuntu/.ansible/tmp/ansible-tmp-1764944139.550931-3815-172802266990579/source",
  "state": "file",
  "uid": 1000
}

```

Abbildung 10: Copy Command Execution

Now we run it against all servers to compare the output.

```

ansible all -m copy -a "src=testfile.txt dest=/home/ubuntu/testfile.txt mode=600"

```

```

ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible all -m copy -a "src=testfile.txt dest=/home/ubuntu/testfile.txt mode=600"
server2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "checksum": "b56df8ed5365fca1419818aa384ba3b5e7756047",
  "dest": "/home/ubuntu/testfile.txt",
  "gid": 1000,
  "group": "ubuntu",
  "mode": "0600",
  "owner": "ubuntu",
  "path": "/home/ubuntu/testfile.txt",
  "size": 20,
  "state": "file",
  "uid": 1000
}
server1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "checksum": "b56df8ed5365fca1419818aa384ba3b5e7756047",
  "dest": "/home/ubuntu/testfile.txt",
  "gid": 1000,
  "group": "ubuntu",
  "mode": "0600",
  "owner": "ubuntu",
  "path": "/home/ubuntu/testfile.txt",
  "size": 20,
  "state": "file",
  "uid": 1000
}

```

Abbildung 11: Idempotency Check

Server1 shows SUCCESS(green) as the file exists, while Server2 shows "CHANGED"(yellow). We verify the file presence on the worker.

```
ubuntu@ip-172-31-73-159:~$ ls -la
total 44
drwxr-x--- 6 ubuntu ubuntu 4096 Dec  5 14:15 .
drwxr-xr-x 3 root    root   4096 Dec  5 13:13 ..
drwx----- 3 ubuntu ubuntu 4096 Dec  5 14:11 .ansible
-rw----- 1 ubuntu ubuntu  174 Dec  5 13:56 .bash_history
-rw-r--r-- 1 ubuntu ubuntu  220 Jan  6  2022 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Jan  6  2022 .bashrc
drwx----- 2 ubuntu ubuntu 4096 Dec  5 13:43 .cache
drwxrwxr-x 3 ubuntu ubuntu 4096 Dec  5 13:53 .local
-rw-r--r-- 1 ubuntu ubuntu  807 Jan  6  2022 .profile
drwx----- 2 ubuntu ubuntu 4096 Dec  5 13:54 .ssh
-rw----- 1 ubuntu ubuntu   20 Dec  5 14:15 testfile.txt
```

Abbildung 12: File on Worker

Finally, we remove the file.

```
ansible all -m file -a "path=/home/ubuntu/testfile.txt_state=absent"
```

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible all -m file -a "path=/home/ubuntu/testfile.txt state=absent"
server2 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": true,
  "path": "/home/ubuntu/testfile.txt",
  "state": "absent"
}
server1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": true,
  "path": "/home/ubuntu/testfile.txt",
  "state": "absent"
}
```

Abbildung 13: Removing File

1.5 Playbooks (The Webserver)

1.5.1 Create Index File

We create a custom index file on the Master.

```
echo "Hello_World_from_Ansible" > index.html
```

1.5.2 Create Playbook

We create the `webserver.yaml` playbook.

```
---
- hosts: all
  become: true

  tasks:
    - name: Install latest version of Apache
      apt:
        name: apache2
        state: latest
        update_cache: yes

    - name: Replace standard index file in /var/www/html/index.html with
      custom index.html
      copy:
        src: index.html
        dest: /var/www/html/index.html

    - name: Restart Apache
      service:
        name: apache2
        state: restarted
      notify: Check HTTP Service

  handlers:
    - name: Check HTTP Service
      uri:
        url: http://{{ ansible_default_ipv4.address }}
        status_code: 200
```

```
Master
GNU nano 6.2 webserver.yaml *
- hosts: all
  become: true

  tasks:
    - name: Install latest version of Apache
      apt:
        name: apache2
        state: latest
        update_cache: yes

    - name: Replace standard index file in /var/www/html/index.html with custom index.html
      copy:
        src: index.html
        dest: /var/www/html/index.html

    - name: Restart Apache
      service:
        name: apache2
        state: restarted
      notify: Check HTTP Service

  handlers:
    - name: Check HTTP Service
      uri:
        url: http://{{ ansible_default_ipv4.address }}
        status_code: 200
```

Abbildung 14: Webserver Playbook

1.5.3 Run Playbook

First, we run against Node 1 only using limits.

```
ansible-playbook webserver.yaml --limit servers1
```

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ echo "Hello World from Ansible" > index.html

ubuntu@ip-172-31-68-127:~/ansible-lab$ nano webserver.yaml

ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible-playbook webserver.yaml --limit servers1

PLAY [all] *****
TASK [Gathering Facts] *****
ok: [server1]

TASK [Install latest version of Apache] *****
changed: [server1]

TASK [Replace standard index file in /var/www/html/index.html with custom index.html] *****
changed: [server1]

TASK [Restart Apache] *****
changed: [server1]

RUNNING HANDLER [Check HTTP Service] *****
ok: [server1]

PLAY RECAP *****
server1 : ok=5  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Abbildung 15: Running on Worker 1

Then, we run against all nodes.

```
ansible-playbook webserver.yaml
```

```
ubuntu@ip-172-31-68-127:~/ansible-labs$ ansible-playbook webserver.yaml

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]

TASK [Install latest version of Apache] *****
ok: [server1]
changed: [server2]

TASK [Replace standard index file in /var/www/html/index.html with custom index.html] *****
ok: [server1]
changed: [server2]

TASK [Restart Apache] *****
changed: [server1]
changed: [server2]

RUNNING HANDLER [Check HTTP Service] *****
ok: [server1]
ok: [server2]

PLAY RECAP *****
server1      : ok=5    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2      : ok=5    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Abbildung 16: Running on All Workers

1.5.4 Verify Deployment

We verify the deployment by accessing the web server in a browser.

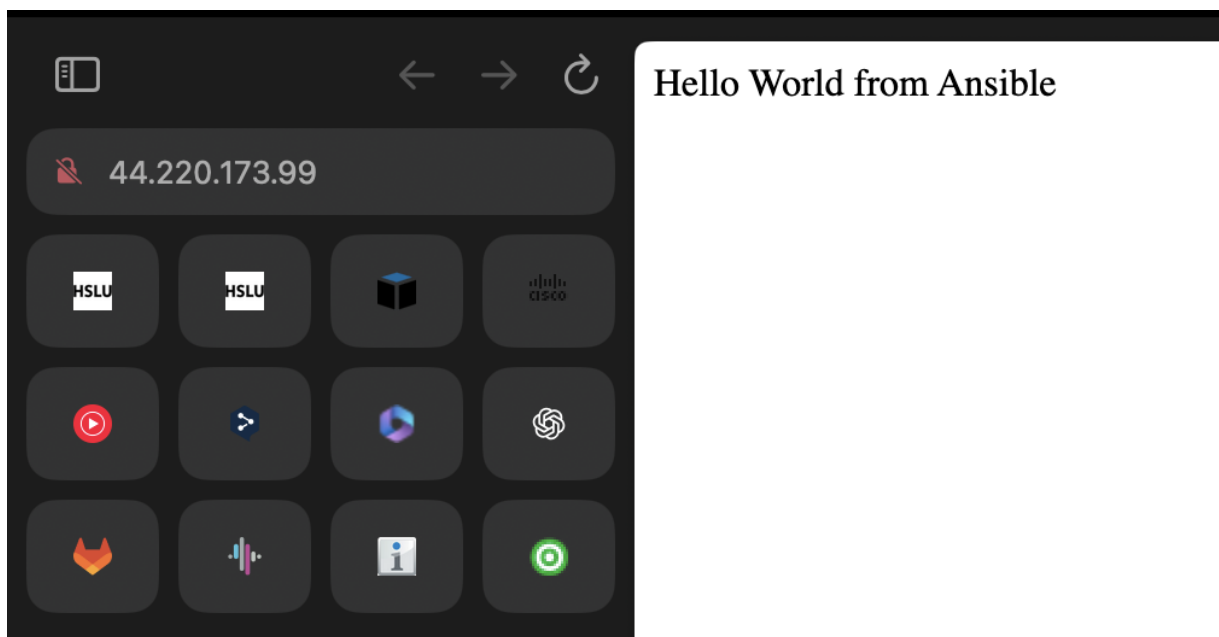


Abbildung 17: Browser Verification

2 Ansible Workshop 2

This second workshop builds upon the previous one. We will refactor our existing playbook into roles, add a MySQL role, and implement tests.

2.1 Prerequisites Check

Detailed checks to ensure the environment is ready.

Login to AWS Academy and switch to the EC2 Dashboard. Make sure your EC2 instances from the first Ansible workshop are online and reachable via SSH. On the master, verify connectivity to the nodes:

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible all -m ping -o
server2 | SUCCESS => {"ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},"changed": false,"ping": "pong"}
server1 | SUCCESS => {"ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},"changed": false,"ping": "pong"}
```

Abbildung 18: Verifying Connectivity

2.2 Refactor Webserver to a Role

We refactor the existing playbook into a reusable role.

First, we create the roles directory and initialize the webserver role:

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ mkdir roles

ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible-galaxy init roles/webserver
- Role roles/webserver was created successfully
```

Abbildung 19: Creating Webserver Role

We move the tasks to `roles/webserver/tasks/main.yml`:

```
GNU nano 6.2
---
# tasks file for roles/webserver
- name: Install latest version of Apache
  apt:
    name: apache2
    state: latest
    update_cache: yes

- name: Replace standard index file
  copy:
    src: index.html
    dest: /var/www/html/index.html

- name: Restart Apache
  service:
    name: apache2
    state: restarted
```

Abbildung 20: Webserver Role Tasks

The `index.html` file is moved to `roles/webserver/files/`.

2.3 Create the MySQL Role

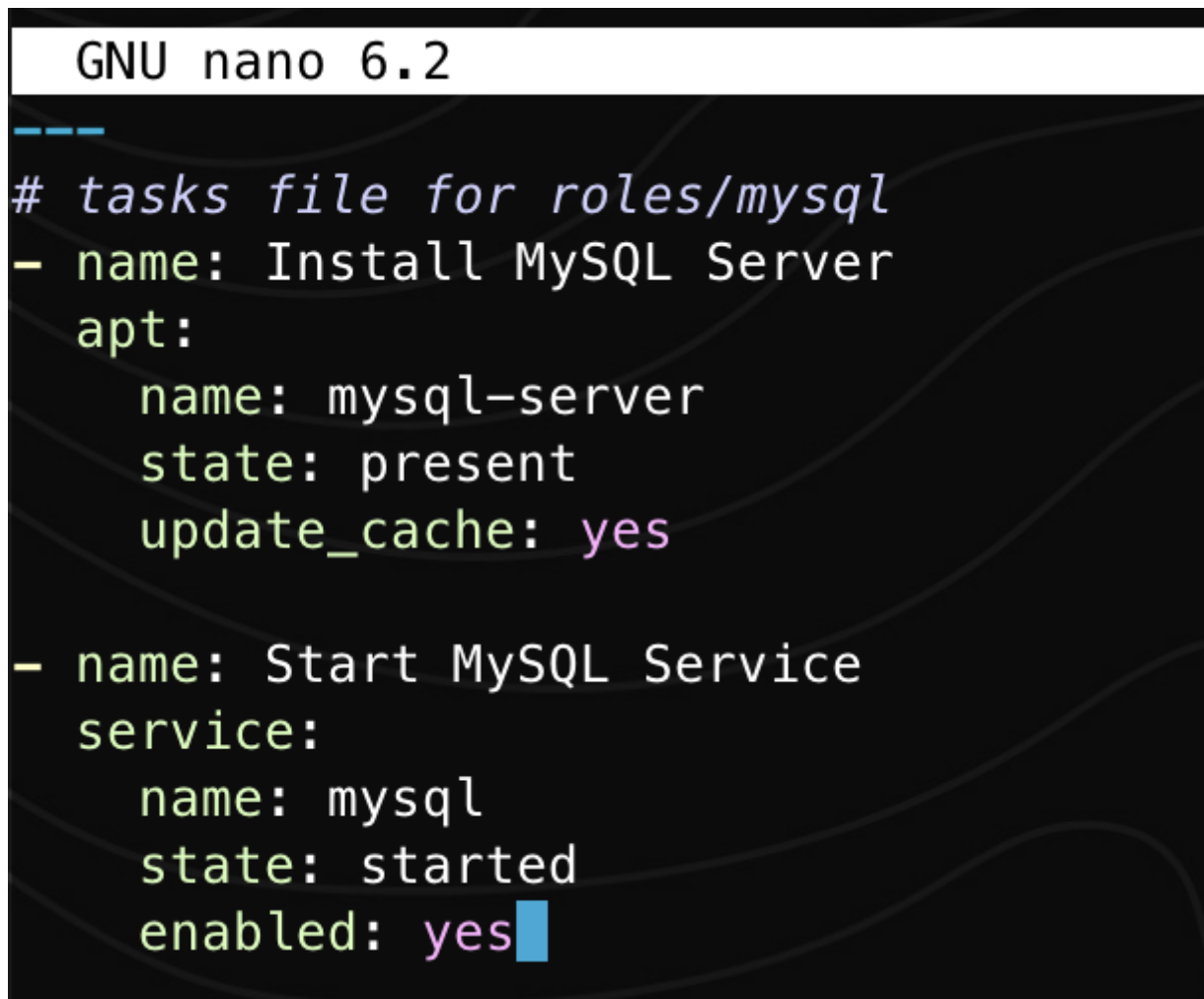
We create a second role for the database.

Initialize the MySQL role:

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible-galaxy init roles/mysql
- Role roles/mysql was created successfully
```

Abbildung 21: Creating MySQL Role

Define tasks in `roles/mysql/tasks/main.yml`:

A screenshot of a terminal window with a black background and a white title bar that reads "GNU nano 6.2". The terminal shows a YAML file for Ansible tasks. The first line is a comment: "# tasks file for roles/mysql". There are two task entries, each preceded by a hyphen. The first task is "name: Install MySQL Server", followed by an indented "apt:" section with "name: mysql-server", "state: present", and "update_cache: yes". The second task is "name: Start MySQL Service", followed by an indented "service:" section with "name: mysql", "state: started", and "enabled: yes". A blue cursor is visible at the end of the "enabled: yes" line.

```
GNU nano 6.2
---
# tasks file for roles/mysql
- name: Install MySQL Server
  apt:
    name: mysql-server
    state: present
    update_cache: yes

- name: Start MySQL Service
  service:
    name: mysql
    state: started
    enabled: yes
```

Abbildung 22: MySQL Role Tasks

2.4 Create the Main Playbook

We create a master playbook `site.yml` to utilize both roles.

GNU nano 6.2

```
---  
- hosts: all  
  become: true  
  
  roles:  
    - webserver  
    - mysql
```

Abbildung 23: Site Playbook Configuration

2.5 Create Tests (TDD)

We verify the roles using a test playbook `test_infrastructure.yaml`.

```
GNU nano 6.2
---
- hosts: all
  become: true

  tasks:
    # 1. Gather facts about all running services
    - name: Populate service facts
      service_facts:

    # 2. Test Apache
    - name: Check if Apache is running
      assert:
        that:
          - "'apache2.service' in ansible_facts.services"
          - "ansible_facts.services['apache2.service'].state == 'running'"
        fail_msg: "Apache is not running!"
        success_msg: "Apache is running correctly."

    # 3. Test MySQL
    - name: Check if MySQL is running
      assert:
        that:
          - "'mysql.service' in ansible_facts.services"
          - "ansible_facts.services['mysql.service'].state == 'running'"
        fail_msg: "MySQL is not running!"
        success_msg: "MySQL is running correctly."
```

Abbildung 24: Test Infrastructure Playbook

2.6 Execution & Verification

Run the main playbook:

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible-playbook site.yaml

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]

TASK [webserver : Install latest version of Apache] *****
ok: [server1]
ok: [server2]

TASK [webserver : Replace standard index file] *****
ok: [server1]
ok: [server2]

TASK [webserver : Restart Apache] *****
changed: [server1]
changed: [server2]

TASK [mysql : Install MySQL Server] *****
changed: [server2]
changed: [server1]

TASK [mysql : Start MySQL Service] *****
ok: [server1]
ok: [server2]

PLAY RECAP *****
server1 : ok=6  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
server2 : ok=6  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Abbildung 25: Running Site Playbook

Run the test playbook:

```
ubuntu@ip-172-31-68-127:~/ansible-lab$ ansible-playbook test_infrastructure.yaml

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]

TASK [Populate service facts] *****
ok: [server1]
ok: [server2]

TASK [Check if Apache is running] *****
ok: [server1] => {
  "changed": false,
  "msg": "Apache is running correctly."
}
ok: [server2] => {
  "changed": false,
  "msg": "Apache is running correctly."
}

TASK [Check if MySQL is running] *****
ok: [server1] => {
  "changed": false,
  "msg": "MySQL is running correctly."
}
ok: [server2] => {
  "changed": false,
  "msg": "MySQL is running correctly."
}

PLAY RECAP *****
server1 : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
server2 : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Abbildung 26: Running Tests